

# Report 6: Fine-Tuning Analysis — Will Domain Adaptation on AVSpeech Improve VSP-LLM?

**Document Classification:** Technical Research Report **Date:** February 18, 2026 **Scope:** Whether and how fine-tuning/QLoRA on diverse YouTube-like data (AVSpeech) would improve the model's real-world performance **Current Performance Baseline:** Mean WER 67.0%, Corpus WER 125.5% on english\_1k (860 segments)

## 1. Executive Summary

The VSP-LLM model's 67% WER on real-world YouTube content (vs. 25.4% on the LRS3 benchmark) is primarily caused by **domain mismatch** — the model was trained exclusively on curated TED talks. Fine-tuning on diverse, YouTube-sourced data like AVSpeech would address this directly.

**Expected improvement: 15-25 WER points** (from ~67% to ~42-52%), with the strongest gains coming from unfreezing the AV-HuBERT visual encoder, which learns to handle diverse head poses, lighting conditions, and speaker appearances. However, fine-tuning is **not a complete solution** — hallucination (an architectural issue) and short-segment failure (a context issue) persist regardless of training data.

## 2. Architecture: What Gets Trained

The VSP-LLM model consists of four trainable components. Understanding what each does is critical for predicting fine-tuning impact.

### 2.1 Component Breakdown

| Component                         | Parameters                      | Trained by Default?                   | Purpose                                          |
|-----------------------------------|---------------------------------|---------------------------------------|--------------------------------------------------|
| AV-HuBERT encoder                 | ~315M                           | Frozen first 18k steps, then unfrozen | Extracts visual features from lip movement video |
| avfeat_to_llm (linear layer)      | 1024 x 4096 = ~4.2M             | Always trainable                      | Maps visual features to LLaMA embedding space    |
| LoRA adapters (Q,K,V projections) | rank 16 x 32 layers x 3 = ~4.2M | Always trainable                      | Adapts LLaMA's attention for lip-reading         |

| Component       | Parameters | Trained by Default?                    | Purpose                      |
|-----------------|------------|----------------------------------------|------------------------------|
| LLaMA-2-7B base | ~6.7B      | <b>Never trained</b> (4-bit quantized) | Language generation backbone |

**Total trainable:** ~323M when encoder unfrozen, ~8.4M when frozen.

## 2.2 Current Training Data

All components were trained on **LRS3** (Afouras et al., 2018): - **433 hours** of TED and TEDx talks - **Curated conditions:** frontal-facing speakers, studio lighting, professional cameras - **Standard vocabulary:** academic English, well-articulated - **Clean audio-derived transcriptions:** high-quality labels

## 2.3 The Domain Gap

The english\_1k test data (YouTube) differs from LRS3 (TED) in every dimension:

| Dimension         | LRS3 (Training)      | english_1k (Test)                  | Impact on WER                                     |
|-------------------|----------------------|------------------------------------|---------------------------------------------------|
| Head pose         | Frontal, static      | Variable, moving                   | High — encoder never learned non-frontal features |
| Lighting          | Professional studio  | Variable, often poor               | High — encoder features degrade                   |
| Speaker diversity | TED presenters       | Global YouTube creators            | Medium — lip shapes vary                          |
| Vocabulary        | Academic English     | Casual, slang, technical, brands   | Medium — LLM generates wrong words                |
| Speech style      | Rehearsed, clear     | Natural, fast, mumbled             | Medium — temporal patterns differ                 |
| Video quality     | Professional cameras | Webcams, phones, screen recordings | Medium — encoder resolution sensitivity           |

## 3. Expected Impact by Component

### 3.1 LoRA Adapters Alone (~4.2M trainable params)

These small adapter matrices shift LLaMA's attention patterns for lip-reading. Training them on diverse data would: - **Help:** Broader vocabulary prior (casual speech, brand names, slang) - **Help:** Better handling of sentence fragments and informal structure - **Not help:** Visual feature

quality (the encoder is frozen) - **Not help:** Hallucination (fundamentally an LLM architecture issue)

The paper's "freeze" variant (LoRA only, encoder frozen) achieves 26.7% WER on LRS3.

**Expected improvement on out-of-domain data: 3-8 WER points** (from ~67% to ~59-64%)

Rank-16 LoRA has very limited capacity — only ~4.2M parameters adapting a 6.7B model. It can nudge vocabulary priors but cannot fundamentally change language generation behavior.

### 3.2 LoRA + Bridge Layer (~8.4M trainable params)

The bridge layer ( `avfeat_to_llm` , 1024 -> 4096 linear) translates visual features into LLaMA's embedding space. Training it on diverse inputs helps it handle the different feature distributions that non-ideal visual conditions produce.

**Expected improvement: 5-10 WER points** (from ~67% to ~57-62%)

### 3.3 LoRA + Bridge + Encoder Unfrozen (~323M trainable params) — RECOMMENDED

This is the "FT" variant from the paper. The AV-HuBERT encoder learns to extract meaningful features from diverse visual conditions — the primary bottleneck.

**Evidence from the paper:** - Freeze (LoRA only): 26.7% WER on LRS3 - Finetune (LoRA + encoder): 25.4% WER on LRS3 — only 1.3 point gain **in-domain** - On out-of-domain data, the encoder adaptation gain would be **much larger** because the encoder has never seen the target visual conditions

**Expected improvement: 15-25 WER points** (from ~67% to ~42-52%)

### 3.4 Summary: Expected Impact

| Configuration                       | Trainable Params | Expected WER   | Improvement         |
|-------------------------------------|------------------|----------------|---------------------|
| Current (no fine-tuning)            | 0                | 67.0%          | Baseline            |
| LoRA only (freeze)                  | ~4.2M            | ~59-64%        | 3-8 points          |
| LoRA + Bridge (freeze)              | ~8.4M            | ~57-62%        | 5-10 points         |
| <b>LoRA + Bridge + Encoder (FT)</b> | <b>~323M</b>     | <b>~42-52%</b> | <b>15-25 points</b> |

## 4. Is This the Same Training Method as the Paper?

### 4.1 Configuration Comparison

| Parameter        | Paper (Section 4.2)          | This Codebase                                       | Match?                          |
|------------------|------------------------------|-----------------------------------------------------|---------------------------------|
| Visual encoder   | AV-HuBERT Large              | AV-HuBERT Large ( <code>large_vox_iter5.pt</code> ) | YES                             |
| LLM backbone     | LLaMA-2-7B                   | LLaMA-2-7B                                          | YES                             |
| Quantization     | QLoRA (4-bit NF4)            | QLoRA (4-bit NF4, double quant)                     | YES                             |
| LoRA rank        | 16                           | 16                                                  | YES                             |
| LoRA alpha       | 32                           | 32                                                  | YES                             |
| LoRA targets     | Q, V projections             | Q, K, V projections                                 | <b>Slightly different</b>       |
| LoRA dropout     | 5%                           | 5%                                                  | YES                             |
| Learning rate    | 5e-4                         | 5e-4                                                | YES                             |
| Optimizer        | Adam (beta1=0.9, beta2=0.98) | Adam (beta1=0.9, beta2=0.98)                        | YES                             |
| LR schedule      | Tri-stage                    | Tri-stage (10k warmup, 0 hold, 20k decay)           | YES                             |
| Total steps      | 30k (for 433h data)          | 30k                                                 | YES                             |
| Encoder freeze   | 18k steps (FT variant)       | 18k (finetune.yaml)                                 | YES                             |
| GPUs             | 8x RTX 3090 (24 GB each)     | 1x Tesla T4 (16 GB)                                 | <b>NO</b>                       |
| Effective batch  | 1 x 8 GPUs = 8               | 1 x update_freq 8 = 8                               | YES (via gradient accumulation) |
| K-means clusters | 200                          | 200                                                 | YES                             |
| Label smoothing  | Not explicitly stated        | 0.1                                                 | Likely match                    |
| Training data    | LRS3 (433h)                  | AVSpeech (YouTube)                                  | <b>Intentionally different</b>  |

## 4.2 The LoRA Target Difference

The paper targets Q and V projections. The codebase also targets K. This adds ~30% more LoRA parameters but is generally harmless or slightly beneficial. The codebase implementation is a reasonable enhancement.

## 4.3 Recommended Modifications for Domain Adaptation

For fine-tuning on AVSpeech (domain adaptation), three changes to the paper's method are recommended:

**Change 1: Start from existing checkpoint (not from scratch)** - The paper trains from pre-trained AV-HuBERT + fresh LoRA - For domain adaptation, start from `checkpoint_finetune.pt` (already LRS3-trained) - This is standard transfer learning — preserves lip-reading ability, adds new domain

**Change 2: Reduce encoder freeze duration** - Paper: freeze for 18k of 30k steps - Recommended: freeze for **0-5k steps** only - Rationale: The encoder IS the bottleneck for domain adaptation — we want it to adapt as early as possible - The short freeze gives LoRA/bridge time to warm up before encoder gradients propagate

**Change 3: Scale training steps to data size** - Paper: 30k steps for 433h of data - If AVSpeech subset < 100h: **10k-15k steps** to avoid overfitting - Scaling formula: `steps = 30k * (data_hours / 433h)` - Risk: Overfitting on small diverse dataset is worse than underfitting

---

## 5. GPU Requirements: Tesla T4 vs. 8x Large GPU

---

### 5.1 Current Server Hardware

| Component   | Specification                            |
|-------------|------------------------------------------|
| GPU         | Tesla T4 (15,360 MiB / 16 GB VRAM)       |
| GPU Compute | 8.1 TFLOPS FP32, 65 TFLOPS FP16 (Tensor) |
| CPU         | Intel Xeon Platinum 8259CL @ 2.50GHz     |
| RAM         | 30 GB                                    |
| Disk        | 157 GB free                              |

---

## 5.2 Training Memory Budget (T4, 16 GB)

| Component                                 | VRAM Usage (estimated) |
|-------------------------------------------|------------------------|
| LLaMA-2-7B (4-bit quantized)              | ~3.5 GB                |
| AV-HuBERT encoder (FP16)                  | ~0.6 GB                |
| Bridge layer + LoRA adapters              | ~0.05 GB               |
| <b>Subtotal: model weights</b>            | <b>~4.1 GB</b>         |
| Forward activations (batch=1, 500 frames) | ~2-4 GB                |
| Backward gradients (encoder unfrozen)     | ~3-5 GB                |
| Optimizer states (Adam momentums)         | ~1-2 GB                |
| CUDA overhead                             | ~1-2 GB                |
| <b>Total estimated</b>                    | <b>~11-17 GB</b>       |

## 5.3 T4 Feasibility

| Phase                   | Fits in 16 GB?         | Notes                                               |
|-------------------------|------------------------|-----------------------------------------------------|
| Encoder frozen          | YES (comfortable)      | ~8-10 GB total                                      |
| Encoder unfrozen        | <b>TIGHT</b> (may OOM) | ~14-17 GB, may need max_sample_size reduction       |
| Estimated training time | ~25-50 hours           | Very slow due to single GPU + gradient accumulation |

## 5.4 8x Large GPU Instance (RECOMMENDED)

| Instance     | GPUs    | VRAM/GPU | Total VRAM | Cost/hr  | Training Time |
|--------------|---------|----------|------------|----------|---------------|
| p3.16xlarge  | 8x V100 | 16 GB    | 128 GB     | ~\$24/hr | ~3-5 hours    |
| p4d.24xlarge | 8x A100 | 40 GB    | 320 GB     | ~\$32/hr | ~2-3 hours    |
| g5.48xlarge  | 8x A10G | 24 GB    | 192 GB     | ~\$16/hr | ~3-4 hours    |
| p5.48xlarge  | 8x H100 | 80 GB    | 640 GB     | ~\$98/hr | ~1-2 hours    |

## 5.5 Why 8x GPU Is Dramatically Better

1. **Matches the paper's exact setup** — the config already has `distributed_world_size: 8`. No code changes needed.
2. **No memory constraints** — 24+ GB per GPU means unfrozen encoder fits comfortably

3. **Training time: ~3-5 hours vs. ~25-50 hours** — enables iterating on multiple experiments per day
4. **Cost: ~\$120-200 per run** on p3.16xlarge — cheaper than 2 days of continuous T4

## 5.6 Recommended Workflow

| Task                            | Where to Run           | Why                                   |
|---------------------------------|------------------------|---------------------------------------|
| Data preprocessing (stages 1-7) | <b>T4 instance</b>     | CPU-bound, T4 handles it fine         |
| Model training (stage 8)        | <b>8x GPU instance</b> | Multi-GPU for speed, no memory limits |
| Evaluation / decode             | <b>T4 instance</b>     | Single-GPU decode works fine          |
| K-means re-clustering           | <b>Either</b>          | GPU helps but not critical            |

The config only needs one change for 8x GPU:

```
optimization:
  update_freq: [1]    # Change from [8] – no accumulation needed with 8 real GPUs
```

## 6. The AVSpeech Data Challenge

### 6.1 AVSpeech Dataset Overview

AVSpeech (Ephrat et al., Google, SIGGRAPH 2018) is a large audio-visual dataset: - ~4,700 hours from ~290,000 YouTube videos - Segments of 3-10 seconds, single visible speaker - Wide variety of people, languages, face poses - **No transcriptions provided** — designed for speech separation, not recognition

### 6.2 Generating Training Labels

Since VSP-LLM trains with (video, transcription) pairs, AVSpeech needs transcriptions. The best approach:

1. **Run Whisper ASR on AVSpeech audio** to generate transcriptions
2. This is already built into the pipeline (Stage 4: `lib/asr.sh`)
3. Quality: Whisper achieves ~5-10% WER on clean speech, higher on noisy
4. **This creates a noise ceiling** — the lip-reading model cannot exceed label quality

### 6.3 Is Noisy Labeling Acceptable?

**Yes, for domain adaptation.** The key insight:

Noisy labels from many domains > perfect labels from one domain

The current model has perfect LRS3 labels but catastrophically fails on YouTube because it never saw diverse visual conditions. Even with 10% label noise from Whisper, the visual encoder would learn to handle: - Side profiles and moving heads - Poor lighting and webcam quality - Diverse lip shapes and speaking styles

This is validated by LiteVSR (ICASSP 2024), which achieved competitive results using only Whisper-generated labels for training.

## 7. What Fine-Tuning Will and Won't Fix

### 7.1 Failure Modes Addressed

| Failure Mode                                   | Current Impact   | Helped by Fine-Tuning?  | Explanation                                                            |
|------------------------------------------------|------------------|-------------------------|------------------------------------------------------------------------|
| <b>Visual domain mismatch</b>                  | ~all segments    | <b>YES (strongly)</b>   | Encoder learns diverse visual conditions                               |
| <b>Vocabulary mismatch</b>                     | ~60% of segments | <b>YES (moderately)</b> | LoRA learns YouTube vocabulary                                         |
| <b>Named entity errors</b><br>(F1: 38.8%)      | 85% of segments  | <b>Partially</b>        | More diverse vocabulary in training data, but proper nouns remain hard |
| <b>Insertion errors</b><br>(Corpus WER 125.5%) | Widespread       | <b>Partially</b>        | Better visual features reduce ambiguity, fewer hallucinated insertions |

### 7.2 Failure Modes NOT Addressed

| Failure Mode                             | Current Impact    | Helped by Fine-Tuning? | Why Not                                                                                                               |
|------------------------------------------|-------------------|------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Hallucination</b><br>(WER >= 100%)    | 20.6% of segments | <b>No</b>              | Architectural — the LLM generates from language prior when visual features are ambiguous, regardless of training data |
| <b>Short segment failure</b> (1-5 words) | ~8% of segments   | <b>No</b>              | Architectural — the LLM needs context length; more training data doesn't add context at inference time                |



| Failure Mode               | Current Impact | Helped by Fine-Tuning? | Why Not                                                                                           |
|----------------------------|----------------|------------------------|---------------------------------------------------------------------------------------------------|
| <b>Homophene ambiguity</b> | Fundamental    | <b>Minimally</b>       | Many words look identical on the lips (/p/, /b/, /m/). No amount of training changes lip physics. |

### 7.3 Realistic Outcome

| Metric                     | Current | After Fine-Tuning (est.) | Improvement        |
|----------------------------|---------|--------------------------|--------------------|
| Mean WER                   | 67.0%   | ~42-52%                  | 15-25 points       |
| Corpus WER                 | 125.5%  | ~85-105%                 | 20-40 points       |
| Perfect (WER=0)            | 1.9%    | ~4-8%                    | 2-4x more          |
| Good (WER <= 20%)          | 11.4%   | ~18-28%                  | 2-2.5x more        |
| Catastrophic (WER >= 100%) | 20.6%   | ~12-18%                  | Moderate reduction |
| NEA F1                     | 38.8%   | ~45-55%                  | 6-16 points        |

## 8. Alternative and Complementary Approaches

### 8.1 Approaches That Complement Fine-Tuning

These can be combined with fine-tuning for cumulative improvement:

| Approach                         | Est. WER Reduction  | Effort | Combined with FT?   |
|----------------------------------|---------------------|--------|---------------------|
| Hyperparameter tuning (Report 2) | 10-15 points        | Low    | Yes (independent)   |
| Prompt engineering (Report 3)    | 5-10 points         | Low    | Yes (independent)   |
| Confidence scoring (Report 4)    | Enables filtering   | Medium | Yes (post-training) |
| N-best aggregation (Report 5)    | 5-10 points         | Medium | Yes (post-training) |
| <b>All above + fine-tuning</b>   | <b>25-40 points</b> | High   | Cumulative          |

## 8.2 Alternative Training Approaches (Literature)

| Method                         | Source                           | Data Required        | Expected Impact                                  |
|--------------------------------|----------------------------------|----------------------|--------------------------------------------------|
| Whisper knowledge distillation | Prajwal et al., Interspeech 2024 | Unlabeled AV data    | 5-10 WER points (complementary to QLoRA)         |
| Speaker-adaptive prompt tuning | Shin et al., AAAI 2025           | 5 min per speaker    | Significant per-speaker gains                    |
| MultiVSR dataset fine-tuning   | Prajwal et al., ICASSP 2025      | 12k hrs labeled      | Potentially stronger than AVSpeech (pre-labeled) |
| VALLR phoneme decomposition    | Thomas et al., ICCV 2025         | Architectural change | Addresses hallucination at architecture level    |

## 9. Implementation Roadmap

### Phase 1: Data Preparation (This T4 instance, ~1-2 days)

1. Transfer AVSpeech videos to this instance ( `/home/ubuntu/vsp_input/` )
2. Run existing pipeline stages 1-7 (normalize, segment, crop, ASR, manifests, clustering)
3. Verify training data format (train.tsv, train.wrd, train.km, train.cluster\_counts)

### Phase 2: Training (8x GPU instance, ~3-5 hours)

1. Transfer processed data to 8x GPU instance
2. Run fine-tuning with AVSpeech config:
3. Start from `checkpoint_finetune.pt`
4. Freeze encoder for 5k steps, unfreeze for 10k steps
5. `update_freq: [1]` with 8 real GPUs
6. Save best checkpoint based on accuracy metric

### Phase 3: Evaluation (This T4 instance, ~2-4 hours)

1. Transfer fine-tuned checkpoint back
2. Re-run decode on english\_1k with new checkpoint
3. Generate report, compare WER/NEA against baseline
4. If improvement is sufficient, deploy; otherwise iterate

### Phase 4: Post-Training Optimization (optional, ~1 day)

1. Re-run K-means clustering with fine-tuned encoder features

2. Regenerate cluster counts
  3. Re-decode with new clusters
  4. Apply hyperparameter tuning from Report 2
- 

## 10. Conclusion

---

Fine-tuning on AVSpeech data is **the right strategic move** for improving real-world performance. The expected 15-25 WER point improvement (from ~67% to ~42-52%) would make the model meaningfully more useful, though still far from perfect.

Key decisions: - **Unfreeze the encoder** — this is where 70% of the gain comes from - **Use 8x GPU instance** for training — 10-20x faster, no memory constraints, matches paper's setup - **Start from existing checkpoint** — transfer learning preserves LRS3 knowledge - **Generate labels with Whisper** — noisy but acceptable for domain adaptation - **Combine with hyperparameter tuning and confidence scoring** for maximum cumulative improvement

The model's fundamental limitation — LLM-based decoding causes hallucination when visual features are ambiguous — cannot be solved by training data alone. For production reliability, confidence scoring (Report 4) and N-best aggregation (Report 5) remain essential regardless of fine-tuning.

---

## 11. References

---

1. Yeo et al. (2024). "VSP-LLM: Visual Speech Processing with LLMs." arXiv:2402.15151.
2. Ephrat et al. (2018). "Looking to Listen at the Cocktail Party." SIGGRAPH 2018. — AVSpeech dataset.
3. Djilali et al. (2024). "Do VSR Models Generalize Beyond LRS3?" WACV 2024. — WildVSR benchmark.
4. Shin et al. (2025). "Personalized Lip Reading." AAAI 2025. — Speaker-adaptive LoRA.
5. Prajwal et al. (2024). "Speech Recognition Models are Strong Lip-readers." Interspeech 2024. — Whisper-to-lip-reading.
6. Prajwal et al. (2025). "Scaling Multilingual Visual Speech Recognition." ICASSP 2025. — MultiVSR dataset.
7. Thomas et al. (2025). "VALLR: Visual ASR Language Model for Lip Reading." ICCV 2025. — Phoneme-centric architecture.
8. Afouras et al. (2018). "LRS3-TED: A Large-Scale Dataset for Visual Speech Recognition." arXiv:1809.00496.
9. Dettmers et al. (2023). "QLoRA: Efficient Finetuning of Quantized LLMs." arXiv: 2305.14314.
10. Guo et al. (2017). "On Calibration of Modern Neural Networks." ICML 2017.